T-MAN: Enabling End-to-End Low-Bit LLM Inference on NPUs via Unified Table Lookup

Jianyu Wei[1,*] **Qingtao Li**[2,*] **Shijie Cao**[2] **Lingxiao Ma**[2] **Zixu Hao**[3] **Yanyong Zhang**[1]
Xiaoyan Hu[4] **Ting Cao**[5,†]
[1]University of Science and Technology of China [2]Microsoft Research [3]Tsinghua University
[4]Microsoft [5]Institute for AI Industry Research, Tsinghua University

arXiv:2511.11248v1 [cs.AR] 14 Nov 2025

Abstract

Large language models (LLMs) are increasingly deployed on customer devices. To support them, current devices are adopting SoCs (System on Chip) with NPUs (Neural Processing Unit) installed. Although high performance is expected, LLM inference on NPUs is slower than its CPU counterpart. The reason is that NPUs have poor performance on computations other than GEMM, like dequantization. Current works either disaggregate prefill on the NPUs and decoding on the CPUs, or put both on the NPUs but with an accuracy loss.

To solve this issue, based on the insight that low-bit can enable target computation encoded within an acceptably sized table, we propose table lookup to subsume hardware operations otherwise unsupported. To realize this, we overcome the conflicting hardware behavior of prefill and decoding to design a unified table layout and tiling through (1) fused two-level table-based dequantization and (2) concurrency-hierarchy-guided tiling. Based on that, we implement the prefill phase by three-stage pipeline and map the table-lookup-based decoding to NPU's vector units. Results show 1.4× and 3.1× speedup for prefill and decoding respectively, and 84% energy savings compared to the baseline NPU methods. The code is available at https://github.com/microsoft/T-MAC/tree/main/t-man.

1 Introduction

Large language models (LLMs) are becoming an integral part of system services on customer devices, to provide unprecedented and always-available AI experiences. Major customer product companies have already deployed LLMs directly in their flagship products, such as Apple Intelligence on iPhone and Mac [8, 9], Google Android on Samsung Galaxy [17], and Microsoft Windows on Copilot PC [25]. Due to strict memory constraints, these deployments universally rely on low-bit weight and activation representations. For example, Apple's foundation models use 2–4 bit [9], Windows Copilot adopts the 4-bit Phi-Silica model [25], and Google's Gemma for mobile runs in 4 bit [17]. More aggressive methods employing 2-bit and even 1-bit precision continue to emerge [13, 36].

Hardware-wise, consumer devices are increasingly equipped with SoCs featuring dedicated NPUs [4] for efficient AI inference, particularly in the LLM era. For example, the Qualcomm Snapdragon SoC with NPU has been used by dozens of different device modules, including Microsoft Surface, Samsung Galaxy, Honor Magic, etc., to support LLMs on devices [33]. Modern NPUs are designed to have the utmost performance and efficiency to run dense GEMMs. Qualcomm NPU is claimed to have 45 TOPS peak performance, 150× of its CPU and 9× of its GPU counterparts [32].

However, despite their impressive peak computation performance, low-bit decoding on NPUs is often slower than on CPUs as other work measured [38]. The fundamental issue is that the utmost efficiency of NPU comes at the cost of flexibility. It is specifically optimized for dense GEMM with specific numerical representation (e.g., INT8) and quantization granularity (e.g., channel-wise), but very poor performance for others. For the diverse low-bit LLM format, when it does not match the native NPU GEMM format, the weights must be dequantized online. This dequantization involves element-wise float-point operations, an operation pattern for which NPUs are not optimized.

Current practice addresses this mismatch in two unsatisfactory ways: (1) *To align LLM quantization with hardware supported format [31], but with an accuracy loss.* It avoids runtime dequantization, but our evaluation shows that adopting hardware per-channel quantization causes a 1.45× worse perplexity, compared to standard quantization methods. (2) *To run the prefill phase on the NPU and decoding phase on the CPUs [40], but with energy overhead.* The prefill is GEMM-dominant, which allows the NPU's GEMM throughput to offset the dequantization cost. The decoding is memory-intensive GEMV-dominant, so the dequantization overhead prevails. However, decoding on CPU consumes 79% more energy and competes with other applications co-run on the customer devices.

This paper aims to achieve the most efficient LLM inference on customer device SoCs, by enabling end-to-end inference on the NPU with superior performance for both prefill and decoding without sacrificing accuracy. Our *key insight* is that if the target computation can be encoded within an acceptably sized table, a lookup-table (LUT) mechanism can subsume hardware operations otherwise unsupported. Low-bit quantization exponentially shrinks the space of possible

* Jianyu Wei and Qingtao Li have equal contributions to this article.
† Corresponding to Ting Cao <tingcao@mail.tsinghua.edu.cn>.

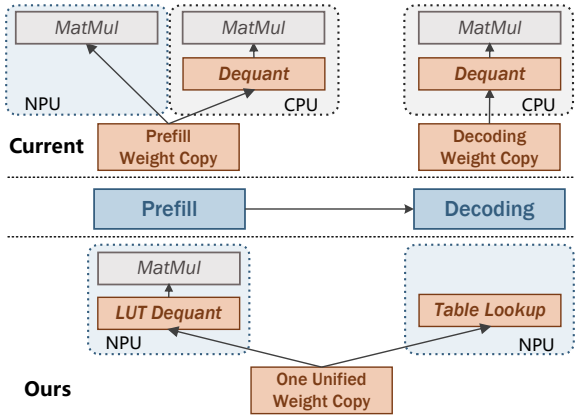


Figure 1. T-MAN versus current practice. To maintain accuracy, current practice offloads decoding phase to CPU and store two weight copies for NPU and CPU respectively. T-MAN leverages table lookup to enable both prefill and decoding on the NPU and only keep one weight copy.

values, making the requisite tables small enough to reside in on-chip memory. Based on this insight, this paper explores table lookup to bridge the gap between flexible quantization and NPU-specialized execution by eliminating the runtime dequantization cost, as illustrated in Fig. 1.

Though the idea is clear, directly applying it on NPUs faces two major challenges. (1) **The conflict of table data layout between prefill and decoding**. In prefill phase, it is necessary to dequantize each low-bit weight to the hardware-specialized GEMM, which requires the bit-parallel table data layout. While for decoding, to eliminate dequantization needs bit-serial data layout to control the table size. Naïvely supporting both doubles on-chip storage. (2) **The conflict of table lookup tiling of prefill and decoding**. The matrix cores for prefill and vector cores for decoding have different instruction widths and computation patterns, leading to different loop orders and tile sizes. This prevents contiguous storage access across both stages.

We propose two techniques accordingly to realize a unified table layout and tiling scheme for both prefill and decoding: (1) **Two-level table lookup** to unify the bit-parallel and bit-serial data layout. (2) **Concurrency-hierarchy-guided tiling scheme** to unify the tiling of table lookup between prefill and decoding. Based on the unified data layout and tiling, we realize the prefill phase by a DMA-vector-matrix three-phase pipeline to parallely conduct dequantization and matrix multiplication, and realize decoding phase by mapping the LUT-based decoding on vector cores of NPU.

We implement T-MAN based on ExecuTorch [7] and evaluate it on mobile devices with Llama 3, Qwen 3 and BitNet with four different bit formats. Compared to the SOTA LLM inference methods on device (including llm.npu [40], T-MAC [38], and llama.cpp [2], bitnet.cpp [37]), we achieve up to 1.4× speedup for prefill and 3.1× for decoding, and up to 84% energy saving compared to the current SOTA NPU solution.

Our contributions can be summarized as follows:

- We realize the first end-to-end LLM inference on NPUs with SOTA speed and energy efficiency compared to other on-device LLM inference systems.
- We propose the novel fused two-level table lookup method to unify the table layout and the concurrency-hierarchy-guided tiling scheme to unify the tiling for both prefill and decoding.
- Based on the unified data layout and tiling, we realize the DMA-Vector-Matrix pipeline for prefill and map the LUT-based GEMV for decoding.

2 Background and Motivation

2.1 LLMs on Edge and Low-bit Quantization

LLMs are increasingly integrated as a foundation system service to deploy on customer devices, such as Gemini on Android, Apple Intelligence on iOS, and Phi in Windows Copilot, to provide real-time, always-on, and privacy-preserving AI services.

LLM inference consists of two distinct phases: (1) *prefill*, which processes the input prompt in parallel and is highly compute-intensive, and (2) *decoding*, which generates tokens autoregressively and is dominated by memory-bound operations, particularly in the common single-batch scenarios on consumer devices. The challenge becomes more pronounced with emerging reasoning-oriented LLMs [11, 28, 35], which often produce thousands of tokens per response.

To reduce the heavy memory bandwidth demands of decoding, low-bit quantization has become indispensable in real-world deployments. Notable examples include Google's 4-bit Gemma[17], Microsoft's 4-bit Phi[25] and 1.58-bit BitNet[24], and Apple's 2–bit foundation models[8]. Meanwhile, new quantization algorithms with diverse bit-widths and granularities continue to emerge at a rapid pace.

2.2 Low-Bit LLM Inference

There are diverse quantization formats used by different algorithms for various application scenarios [2, 12–14, 23, 30, 39, 41]. The formats can differ in bit widths (e.g., 8-, 4-, 2-, and 1-bit), numerical representation (e.g., FP and INT) and quantization granularity (e.g., group-wise with sizes of 32-, 64-, or 128-, as well as channel- and tensor-wise). Weights and activations normally employ different formats. No single quantization format has emerged as dominant. Contradictorily, hardware designs are impossible to simultaneously support all these formats, due to the limited chip area.

To bridge the gap between the algorithm flexibility and hardware, there are two main methods used currently.

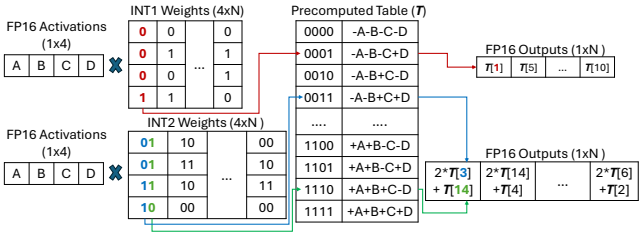


Figure 2. Bit-serial table lookup to implement GEMM. Weight is decomposed into one-bit matrices, serving as the indices to precomputed results stored in tables to look up.

Dequantization-based low-bit inference. Dequantization converts the quantized values back into a higher precision to match the hardware supported formats. The conversion is performed element-wise as: $x_{dequantized} = (x_{quantized} - zero_point) \times scaling_factor$. The resulting dequantized values can then be processed directly by existing hardware units. This approach has been widely adopted for low-bit LLM deployment [2, 3, 22, 40], as it leverages mature hardware for GEMM. However, it introduces extra computation overhead due to the dequantization step.

Bit-serial table lookup. To eliminate the dequantization, recent works, such as LUT-GEMM [29], T-MAC [38], and LUT tensor core [26], propose a unified computation paradigm for diverse low-bit formats, i.e., bit-serial table lookup. As Fig. 2 shows, the key idea is based on the linear equivalent transformation of matrix multiplication: the multiplication of two matrices can be transformed into multiplying one matrix by each one-bit decomposition of the other, followed by shifting and accumulating the partial results. Therefore, the matrix multiplication of diverse quantization formats is reduced to unified operations between the activation matrix and the one-bit weight matrices.

Though current hardware lacks native one-bit computation support, the limited value set (1 and 0) of one-bit weights enables implementation via lookup tables. Each vector pattern of the one-bit weight matrix serves as an index to precomputed results stored in tables, to read the vector results directly. This approach makes inference independent of quantization format and avoids dequantization. The downside, however, is that current NPUs provide much lower-performance support for table lookup instructions compared to GEMMs. It is challenging to achieve practical speedup.

Both approaches have clear trade-offs. Dequantization-based inference fully exploits specialized hardware units but incurs extra overhead from the conversion step. Bit-wise table lookup avoids dequantization and unifies diverse quantization formats but suffers from limited hardware support. In this work, we propose a new method that unifies the two paradigms by leveraging the distinct characteristics of the prefill and decoding phases in LLM inference.

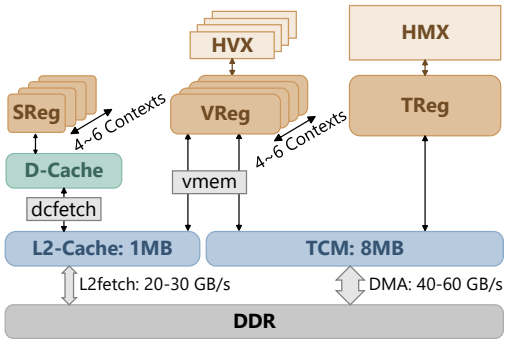


Figure 3. A typical NPU architecture, shown with Snapdragon8 NPU. It integrates the matrix core (HMX), vector cores (HVX), scalar units, and the on-chip memory (TCM).

2.3 NPUs on Mobile SoCs

To support the widely used AI applications on devices, NPUs are designed for utmost performance and efficiency to accelerate neural network inference. These designs emphasize hardware specialization for the most computation-intensive kernels, i.e. dense GEMMs, at certain numerical precisions, while eliminating general-purpose control units that add overheads. Although NPU implementations vary across vendors [4], such as the ones from Huawei and MediaTek, their architecture shares similar design principles.

Snapdragon NPU. We use the Qualcomm Snapdragon NPU as a representative example. Snapdragon NPU features top computation performance among the mobile NPUs and relatively accessible SDKs. They are widely used by diverse flagship mobile devices from Microsoft, Samsung, Xiaomi, Honor, OnePlus etc. The Snapdragon X Elite NPU has 45 TOPS peak performance, much higher compared to the 4.6 TFLOPS GPU and <1 TOPS CPUs on the same SoC [32].

As shown in Fig. 3, the computing units of a Snapdragon NPU consist of one matrix core known as HMX (Hexagon Matrix eXtensions) operating on a 32×32 tile, the 4 to 6 vector cores known as HVX (Hexagon Vector eXtension) with a 1×256 tile, and the scalar computing units.

For the memory hierarchy, each of the computation units has its own dedicated register file. The vector and scalar registers can keep 4 to 6 contexts, enabling 4 to 6 hardware threads to run concurrently. Beneath the registers is the large software-managed on-chip memory, named TCM (Tightly Coupled Memory), with a capacity of 8 MB and a burst transfer width of 2 KB from TCM to HMX, matching the grand computation width for HMX and HVX. There is also the general L2 cache (1 MB size and 128 B access width) shared by the vector and scalar units. Data loading from the off-chip DDR to the TCM is via DMA path which can achieve 40-60 GB/s, while to the L2 cache can achieve 20-30 GB/s (detailed measurements in Table 2).

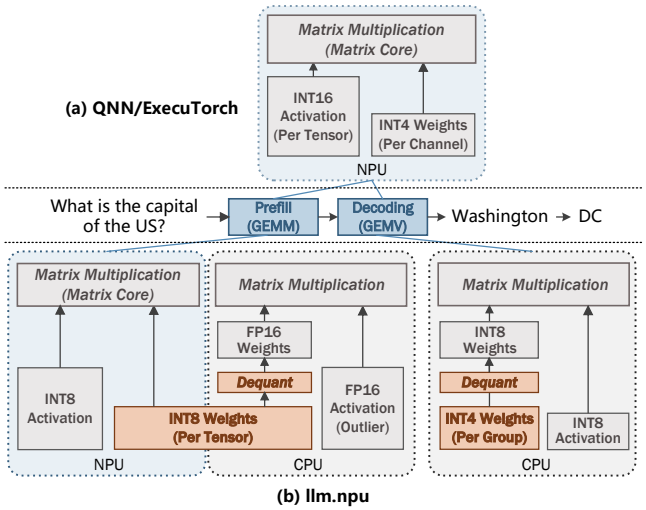


Figure 4. Current practices in leveraging NPUs for low-bit LLM inference, illustrated with SOTA frameworks: Qualcomm QNN from industry and llm.npu from academia.

QNN SDK. The primary SDK for developing on Qualcomm NPUs is the Qualcomm AI Engine, aka the QNN SDK [31]. It is closed source. On-device inference frameworks such as ExecuTorch [7], ONNX Runtime [5], llm.npu [40], and PowerServe [6], leverage QNN as their backend for NPUs. Developers cannot easily customize low-level kernels. To our knowledge, T-MAN is the first to provide low-bit kernels for diverse quantization algorithms for Qualcomm NPUs.

3 Challenges of Low-Bit LLMs on NPUs

As explained in Section 2.2, the main challenge in deploying low-bit LLMs on NPUs still lies in the mismatch between hardware specialization and the diversity of quantization algorithms. As outlined below, each of the current practices has inherent deficiencies that prevent them from fully addressing the challenge.

Slow dequantization. A straightforward way to bridge this gap is through dequantization, which converts arbitrary quantization formats into the NPU's supported format. However, as shown in Fig. 5, which is a W_4A_{16} mixed-precision GEMV kernel of Llama3 decoding. We can see the kernel runs 3.8× slower on the NPU than the CPU, due to the 10× slower dequantization speed. For the prefill stage, which is GEMM-based, the dequantization overhead can be offset by NPU's superior GEMM performance. In contrast, the decoding stage relies on memory-bound GEMV operations, where the powerful matrix core is idle. In this setting, the dequantization step emerges as a dominant source of overhead.

Accuracy loss using hardware format. To avoid the dequantization, some frameworks represented by the Qualcomm QNN enforces quantization match the NPU format,

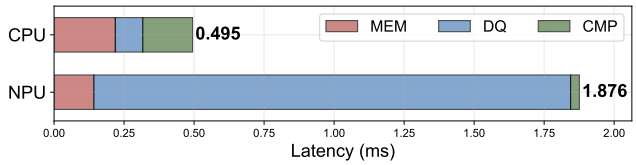


Figure 5. Latency breakdown for a mpGEMV of size 4096 × 4096 × 1, comparing NPU and CPU performance. The total latency is segmented into memory loading (MEM), dequantization (DQ), and computation (CMP).

shown in Fig. 4(a). However, the restricted hardware formats often fail to capture the weight distribution, leading to accuracy loss. As will be presented in Sec. 6.6, using the NPU's per-channel INT4 weight quantization with per-tensor INT16 activation results in 1.45× worse perplexity.

Energy overhead of NPU for prefill and CPU for decoding. To exploit the NPU's GEMM performance without sacrificing accuracy, some frameworks represented by llm.npu only run prefill on the NPU while leaving decoding and outlier computation on the CPU. As shown in Fig. 4(b), during prefill, NPU uses INT8-quantized weights and executes the INT8 GEMM on the matrix core for the best performance. During decoding, the CPU uses INT4-quantized weights, dequantizes them to INT8, and executes INT8 GEMV using SIMD units. While this hybrid approach achieves latency gain, it increases energy consumption by up to 3.4×.

Overall, existing methods fail to achieve simultaneous improvements in latency, energy efficiency, and accuracy when deploying low-bit LLMs on edge NPUs.

4 Design

To enable efficient prefill and decoding on an NPU, we propose a two-pronged approach. For the prefill stage, we employ a dequantization-based GEMM to maximize the use of the NPU's powerful matrix core and maintain flexibility with various quantization algorithms. For the decoding stage, we use a LUT-based GEMV on the less powerful vector core, which eliminates the overhead of dequantization and reduces computational demands.

These distinct computational patterns require weights to be permuted and reshuffled for ideal memory access pattern during execution, and lead to conflicting data layout requirements in three ways: (1) different bit-packing methods (bit-parallel vs. bit-serial), (2) different precision required and (3) mismatched tiling strategies dictated by the computation and register constraints. A naive approach would require two separate model weights, doubling the memory footprint.

To resolve, our key idea is to co-design a single, unified weight layout and tiling strategy that efficiently serves both computation patterns. The overview of the system design is presented in Fig. 6.

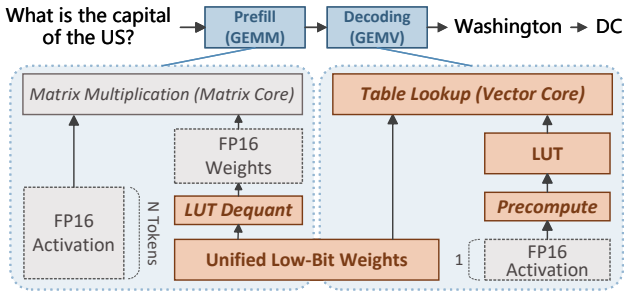


Figure 6. T-MAN system. It runs both prefill and decoding on NPU through table lookup. Only one copy of model weights.

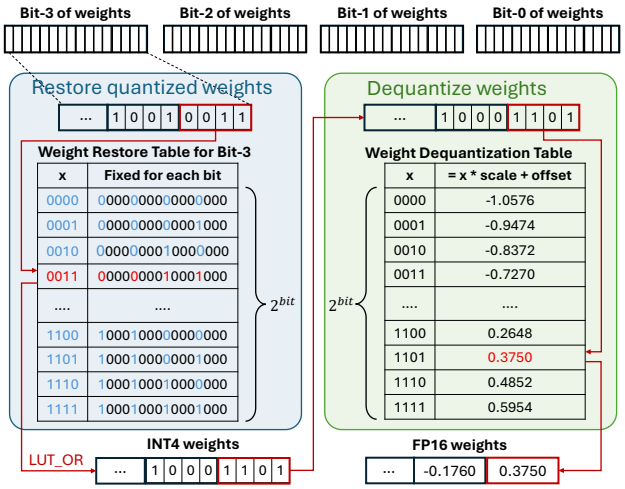


Figure 7. Fused two-level LUT dequantization. Take 4-bit quantized weights as an example.

4.1 Unified Low-Bit Weights for Prefill

Unified Layout through Fused LUT-based Dequantization. Due to the generative, token-by-token nature of the decoding stage, performing any layout transformation introduces significant overhead. Therefore we prioritize the layout required for decoding by using bit-serial packing. Consequently, the bit-repacking is performed during prefill.

However, designing an efficient on-the-fly dequantization process presents two primary challenges: (1) a data layout mismatch exists between the bit-serial nature of LUT-decoding and the bit-parallel required for dequantization, requiring inefficient bit-shuffling operations and (2) NPUs are primarily designed for low power and fast integer operations, feature much slower floating-point conversion and computation, making a naive dequantization implementation the major bottleneck.

To overcome the inefficiency of bit-shuffling and floating-point math operations, we introduce a highly efficient dequantization method that consolidates multiple steps into two distinct LUT operations, as illustrated in Fig. 7.

Bit repacking. A naive approach to repacking data from a bit-serial to a bit-parallel layout involves a slow sequence of bitwise operations (e.g., AND, OR, SHIFT) that handle only one bit at a time. Our method accelerates this process by treating a group of packed bits (for instance, the i-th bit from four different weights) as a direct index into a repacking LUT. The table entry at that index contains the pre-calculated, correctly arranged bit-parallel representation. For example, the packed value 0b0011, representing the most significant bit (MSB) of four INT4 weights, is used to fetch the entry 0b0000 0000 1000 1000. This single lookup instantly places the MSB for each of the four weights into its correct final position. After this is done for all bit positions, the results are combined to reconstruct the original quantized weights. By using a 4-bit LUT index, we replace twelve separate bit manipulation operations (four sets of SHIFT+AND+SHIFT) with just one LUT lookup, achieving a 12× reduction in operations for this step.

Integer-to-float conversion. Instead of performing costly runtime integer-to-float operations, we exploit the small number of unique values in low-bit formats (e.g., 16 for INT4). We pre-calculate the FP16 representation for each of the 16 possible integer values and store them in a conversion LUT.

Applying scales and zero-points. We fuse the affine transformation into a LUT by pre-applying the quantization scales and zero-points to its entries. This "bakes" the transformation directly into the table, which can be shared in the quantization block. For INT2 values and typical quantization block sizes of 64 and 128, this approach reduces the needed floating operations to 4, which is 1/16 and 1/32 of the original respectively. This method effectively removes the main computational bottleneck of the dequantization procedure.

Even with the bit-packing resolved, the mismatched tiling requirements still create memory access overhead during both prefill and decoding. To understand and address the tiling constraints, we first abstract a unified concurrency model that applies to both the vector and matrix cores.

NPU concurrency hierarchy. We analyze the typical NPU's memory hierarchy and SIMD programming model, and summarize the concurrency hierarchy to three levels: (1) *pipeline-level*, where DMA, vector cores, and matrix core execute concurrently, (2) *thread-Level*, where data is processed in parallel by multiple threads on vector cores to compute or prepare for the matrix core, and (3) *SIMD-level*, where data is loaded into vector or matrix registers for SIMD computation.

This hierarchical structure demands a careful tiling design to efficiently fit data into on-chip memory.

Unified tiling strategy. The optimal tiling for dequantization-based prefill differs significantly from that of LUT-based decoding. As illustrated in Fig. 8, for a matrix multiplication of weights (M, K) and activations (N, K), the thread-level tiling and loop orders are:

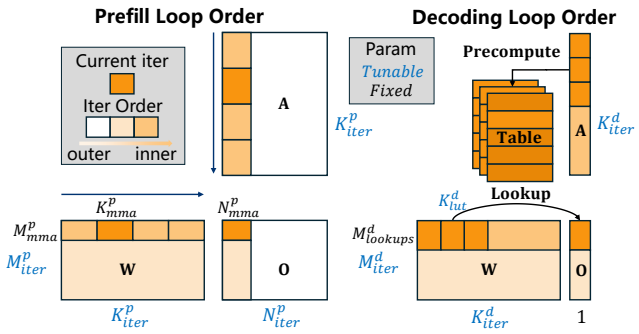


Figure 8. Thread-level tiling and loop orders.

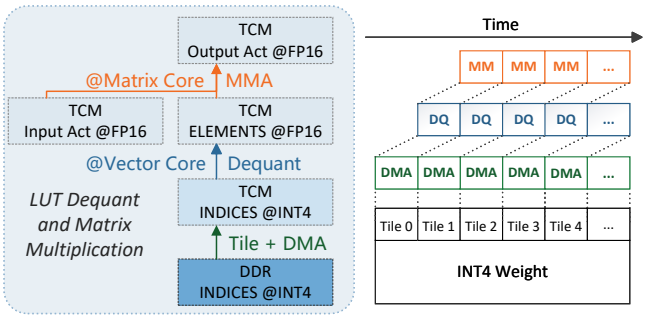


Figure 9. Pipeline of DMA transfer, vector-core dequantization and matrix-core matrix multiplication to hide the latency of memory access and dequantization.

- **Prefill**: $(N^p_{iter}, M^p_{iter}, K^p_{iter}, N^p_{mma}, K^p_{mma}, M^p_{mma})$, where $\{N, M, K\}^p_{iter}$ are tunable iteration counts, $\{N, M, K\}^p_{mma}$ are fixed dimensions of the matrix core's MMA instruction.
- **Decoding**: $(K^d_{iter}, M^d_{iter}, K^d_{lut}, M^d_{lookups})$, where K^d_{iter}, M^d_{iter}, and K^d_{lut} (number of lookup tables in registers) are tunable, while $M^d_{lookups}$ is a fixed number determined by the vector length.

The primary challenge is creating a single, pre-permuted weight layout that serves both tiling schemes. This is critical because weights must be fetched into on-chip memory via DMA in a contiguous block. To find a valid unified tiling, we define a search space following the constraints as:

$$K^d_{lut} < \text{N_REG} \tag{1}$$

$$M^p_{iter} \cdot M^p_{mma} = M^d_{iter} \cdot M^d_{lookups} \tag{2}$$

$$K^p_{iter} \cdot K^p_{mma} = K^d_{iter} \cdot K^d_{lut} \tag{3}$$

$$\text{N_STAGE} \cdot \text{N_THREAD} \cdot S_{tile} < S_{on_chip_mem} \tag{4}$$

$$\text{where } S_{tile} = M^p_{iter} \cdot M^p_{mma} \cdot K^p_{iter} \cdot K^p_{mma}$$

To explain, the tunable parameters are governed by register and TCM capacity. K^d_{lut} must be less than the number of vector registers (Eqn. 1). A unified permutation requires the tile dimensions to match (Eqn. 2, 3), and the total memory footprint of all pipeline stages and parallel threads must not exceed the on-chip memory size (Eqn. 4).

To direct the search, we analyze the characteristics of the NPU, and provide the following heuristics:

- Maximize K^d_{lut} to keep more lookup tables in registers, reducing intermediate write-backs brought by the LUT-based computing pattern as described in Sec. 4.3.
- Maximize M^d_{iter} to increase the data reuse of cached lookup tables.
- Maximize K^p_{iter} to improve the throughput of the matrix core.

Finally, while the thread-level data layout must be contiguous for DMA, the mismatched axis order at the finer-grained SIMD level is acceptable, as non-contiguous loads from TCM to registers do not incur a performance penalty.

4.2 Pipelining to Hide Dequantization Overhead

We can further mask the latency of the dequantization by creating a *DMA-Vector-Matrix three-stage pipeline* that overlaps data movement, data preparation and data consumption. As shown in Fig. 9, our approach pipelines three key stages:

- **DMA transfer:** The DMA engine streams quantized weights from high-latency DDR to the on-chip TCM. The weights are tiled to fit into TCM.
- **Vector-core dequantization:** Concurrently, the vector core dequantizes the previous tile (already in TCM) into FP16 using our fused LUT technique.
- **Matrix-core matrix multiplication:** Simultaneously, the matrix core performs matrix multiplication on the tile prepared in the prior cycle.

This pipelined execution ensures that the memory bus, vector core, and matrix core operate in parallel, maximizing hardware utilization and hiding the dequantization overhead.

4.3 Mapping of LUT-based Decoding on NPU

While dequantization offers flexibility, it creates significant computational overhead and under-utilizes matrix cores during decoding. LUT based method offers a more elegant solution. By treating different data types as table indices and entries, it eliminate the need for dequantization and fully leverage vector cores.

However, unlike traditional dot-product-based methods that vectorize along the input channel (k-axis), LUT-based methods vectorize along the output channel (m-axis). This alternative approach significantly increases the amount of intermediates that must be managed within each computational tile.

As illustrated in Fig.2, at the SIMD level, a single operation involves using a vector of indices to perform many parallel lookups into the same LUT. The results from these lookups belong to different output channels and therefore form a

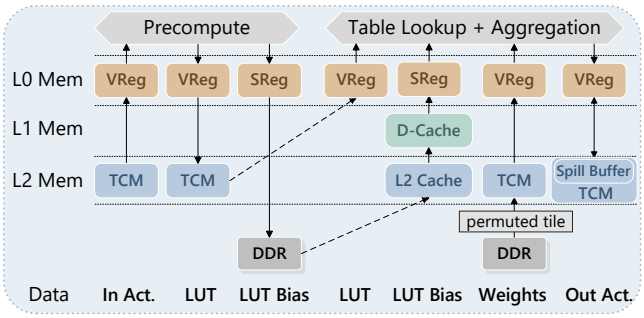


Figure 10. Map LUT decoding memory hierarchy onto NPU.

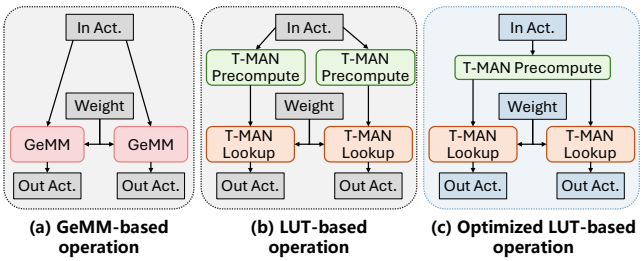


Figure 11. Graph Optimization.

	Bitwidth	CPI	# Look-ups	# Equiv. MADDs
VLUT16	8	0.5	256	1024
	16	0.5	128	512
VLUT32	8	0.5	128	640
	16	0.5	64	320

Table 1. Comparison of VLUT16 and VLUT32 throughput.

vector of partial results, which cannot be immediately aggregated into a single scalar. To handle these large amounts of intermediates, frameworks such as T-MAC align their computation tile size with the quantization block size. This allows all necessary operations—including lookups, partial aggregations, and scaling—to be performed locally within the tile, thereby minimizing costly data movement between the processing unit and memory.

However, this strategy is suboptimal for modern NPUs, which typically have wide vector length (e.g., 1024-bit). Fully exploiting this hardware requires a tile size far larger than a typical quantization block. For example, to optimally use 16 registers reserved for LUTs on Qualcomm NPUs, the tile size on the k-axis needs to be 256, which is much larger than the common quantization block sizes of 64 or 128.

To bridge this gap, T-MAN employs a two-level tiling strategy. The smaller, inner tile aligns with the quantization block for efficient low-precision aggregation. The outer tile is sized to process as many LUTs as can be stored in registers, allowing extensive aggregation of floating-point results before writing to memory. While this maximizes data reuse, it introduces a new bottleneck: the outer tile requires more floating-point accumulators than the hardware register file can hold. This forces the compiler to spill excess registers to the slow L2 cache, severely degrading performance.

To resolve this, we introduce a software-managed register-spill buffer located in the fast, on-chip TCM. This buffer acts as a programmer-controlled extension of the register file, efficiently managing the storage of intermediate accumulators without resorting to high-latency cache.

Fig. 10 depicts the mapping of the LUT decoding onto the NPU memory hierarchy. TCM stores input activations and most intermediates, L2 cache is reserved for scalar values, and the TCM-based spill buffer manages aggregation results.

5 Implementation

T-MAN consists of 7,100 lines of C++ and Python code and is built upon the open-source ExecuTorch library. Key contributions of our implementation include highly optimized LUT-decoding NPU kernels, extended support for custom QNN operator packages in ExecuTorch, and the implementation of more LLMs and quantization schemes.

Graph optimization. LLMs contain layers where the same activation feeds multiple GEMM operations (e.g., Q/K/V projections in attention, up/gate projections in MLPs). One of the practices is to fuse these GEMMs into a larger GEMM, but it is not viable on the NPU, whose limited on-chip memory TCM (8MB) favors splitting them rather than merging them.

For our LUT-based kernels, this pattern leads to redundant computation and excessive memory consumption when the small kernels are scheduled to execute in parallel. We address this by unfusing our kernel into two components: a precomputation kernel and a table look-up kernel. A graph optimization pass then detects shared input patterns and prunes redundant precomputation kernels. As shown in Figure 11, this ensures that multiple table look-up kernels reuse the output from a single precomputation kernel, saving both cycles and memory.

Efficient table look-up by VLUT. To perform efficient table look-ups, T-MAN leverages the VLUT SIMD instruction from HVX. VLUT has two variants: VLUT16, which uses a table of 16 entries with 16 bits per entry, and VLUT32, which uses a table of 32 entries with 8 bits per entry. Both instructions take 8-bit values as indices into the table.

To determine the optimal variant, we analyzed their computational throughput in terms of equivalent Multiply-Adds (MADDs), as detailed in Table 1. It shows that VLUT16 achieves higher throughput for both 8-bit and 16-bit activations. We thus select VLUT16 for our implementation.

Asynchronous DMA. Efficiently transferring data from main memory (DDR) to the processor is critical for performance, especially for the decoding stage. To identify the

Method	Bandwidth (HVX_THREADS=1)	Bandwidth (HVX_THREADS=4)
Vectorized Load	5 GB/s	20 GB/s
L2fetch	26 GB/s	32 GB/s
DMA	59 GB/s	59 GB/s

Table 2. Memory bandwidth microbenchmark on OnePlus 12. (1) **Vectorized Load** implicitly caches data in L2 before loading it to a vector register, (2) **L2 Prefetch** uses the l2fetch instruction to explicitly bring data into the L2 cache, and (3) **DMA** asynchronously transfers data directly from DDR to the NPU's TCM.

optimal method, we benchmarked available approaches provided by the NPU on OnePlus 12 Pro. The results in Table 2 clearly show that DMA achieves the highest and most consistent bandwidth (59 GB/s). The "Vectorized Load" method causes large pipeline stalls due to high memory latency, resulting in the lowest bandwidth.

Based on our findings, we use DMA to load model weights and employ software pipelining to hide memory latency. For smaller data chunks and scalar values that should not be placed in vector registers, we use l2fetch.

6 Evaluation

To demonstrate its efficiency, we evaluate T-MAN using real-world low-bit kernels from a diverse set of LLMs, including Qwen, Llama3, and BitNet. We then compare the end-to-end throughput for both prefill and decoding against leading open-source (llama.cpp [2], llm.npu [40], T-MAC [38], bitnet.cpp [37]) and closed-source (QNN [31]) frameworks. The primary findings of our analysis are summarized as follows:

- Through its efficient LUT-based kernels on the NPU, T-MAN achieves performance advantage even when compared to framework through vendor-specific hardware acceleration like QNN.
- T-MAN supports popular quantization schemes that are unavailable in existing NPU frameworks, including per-group quantization, which is essential for quantization accuracy, and flexible precision combinations such as 2-bit weights.
- By executing all computations exclusively on the power-efficient NPU, T-MAN substantially reduces power consumption compared to both CPU-only and hybrid CPU-NPU solutions.

6.1 Evaluation Setup

Hardware. We evaluate T-MAN on two smartphones: OnePlus 12, equipped with Qualcomm Snapdragon 8 Gen 3 and 24 GB of RAM, and OnePlus 13T, with Qualcomm Snapdragon 8 Elite and 12 GB of RAM.

Models and Quantization. Our evaluation uses popular LLMs: Qwen3-8B [35], Llama-3.1-8B-Instruct [18], and BitNet-2B [24, 36]. We adopt quantization methods from the state-of-the-art research literature [13, 14, 36] that have been evaluated by the deep learning community. Specifically, the Qwen and Llama models are quantized to 4-bit (W_{INT4}) and 2-bit (W_{INT2}) in GPTQ [14] format using an asymmetric, per-block scheme with a block size of 64. Each block shares quantization scales and zero points. Their inference is performed with a $W_{\{INT4,INT2\}}A_{\{INT16,FP16\}}$ setup. BitNet is evaluated with its native per-tensor 1.58-bit weights ($W_{INT1.58}$). Following standard practice, we treat its ternary weights as 2-bit for inference, corresponding to a $W_{INT2}A_{INT16}$ scheme.

Baselines. We compare T-MAN against the following state-of-the-art methods: (1) **llama.cpp:** A widely-used open-source framework for CPU inference on local devices, supporting per-group quantization, (2) **T-MAC:** A state-of-the-art LUT-based kernel library for low-bit CPU inference on local devices, supporting per-group quantization, (3) **bitnet.cpp:** The official inference framework for BitNet, supporting per-tensor quantization, (4) **llm.npu:** An open-source framework designed for NPU acceleration, which uses per-tensor quantization and offloads outliers to the CPU and (5) **QNN:** A proprietary, closed-source framework by Qualcomm. It is highly optimized leveraging internal components like HMX, which are not exposed to developers. It is limited to per-channel and per-tensor quantization.

Kernel-level latency. To ensure consistency, the kernel shapes are taken from the models under evaluation, and the quantization schemes are matched to the source models.

End-to-End throughput. We evaluate end-to-end performance by measuring throughput for a prefill phase with a 1024-token input prompt, followed by a generation phase of 128 tokens, using a batch size of one.

6.2 mpGEMM/mpGEMV Kernel Benchmark

We evaluate the mpGEMM/mpGEMV kernels in Qwen3-8B, Llama-3.1-8B and BitNet-2B on both devices.

mpGEMV. In addition to introducing NPU support for W_{INT2} and per-group quantization, T-MAN achieves significant speedups as shown in Fig. 12.

T-MAN achieves up to 8× speedup compared to QNN-$W_{FP16}A_{FP16}$. llm.npu fails to accelerate the decoding kernel, as high communication costs from offloading outlier calculations force it to fall back to CPU-only kernels.

T-MAN is even 1.8–2.5× faster than QNN for 2-bit kernels and achieves similar performance on 4-bit kernels, despite using a more fine-grained quantization scheme (per-block vs. per-channel). This demonstrates the effectiveness of our software-level optimizations over the framework built upon vendor-specific acceleration.

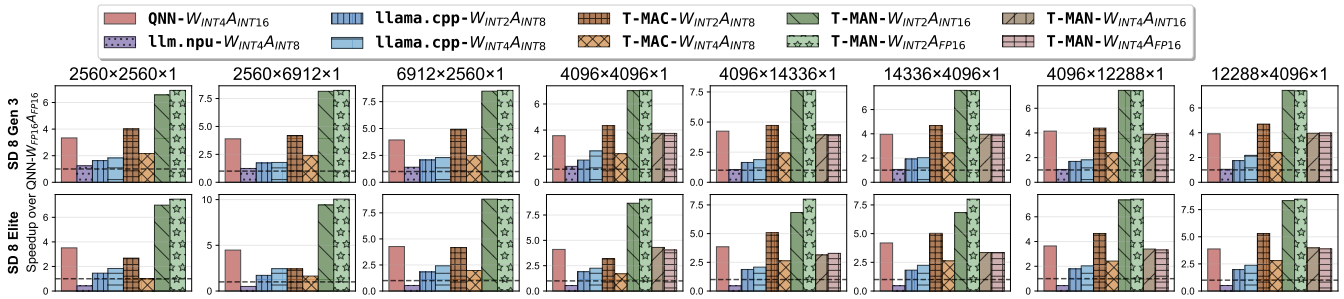


Figure 12. Performance benchmark of the mpGEMV kernel. T-MAN, llama.cpp, and T-MAC use per-block quantization, except for BitNet kernels (shapes {2560,6912}×{2560,6912}) which use per-tensor. QNN is evaluated with per-channel quantization.

mpGEMM. T-MAN dequantizes the per-tensor quantized weights in BitNet kernels to INT8 and the per-block quantized weights in Qwen3-8B and Llama-3.1-8B-Instruct to FP16. The latency of this dequantization step is effectively optimized by LUT-dequantization and pipelining with the matrix multiplication.

As shown in Fig. 13, this approach allows T-MAN to achieve performance comparable to QNN's native $W_{FP16}A_{FP16}$ kernel. Furthermore, T-MAN is considerably faster than llm.npu on smaller matrix shapes (e.g., 2560×2560×128), as it avoids the NPU-CPU synchronization overhead. Attributed to the NPU's high throughput (TOPS), T-MAN delivers a speedup of up to 30× over CPU-only frameworks like llama.cpp and T-MAC.

6.3 End-to-End Inference Throughput

For the decoding phase, T-MAN demonstrates significant performance advantages, achieving a 1.5-1.8× speedup over QNN and a substantial 3.1-3.8× speedup over llm.npu across the tested LLMs. For instance, T-MAN reaches a notable 49.1 tokens/s on BitNet-2B for Snapdragon 8 Gen 3. llm.npu encounters out-of-memory errors on 8B models on the OnePlus 13T with 12 GB RAM, because it needs to store both INT8 weights for prefill and INT4 weights for decoding.

In the prefill stage, T-MAN achieves up to 1.4× speedup compared to llm.npu, and T-MAN-$W_{INT2}A_{FP16}$ achieves similar performance compared to QNN-$W_{FP16}A_{FP16}$ on BitNet. When compared to CPU-based solutions like llama.cpp and T-MAC, T-MAN leverages the high TOPS of NPU to deliver up to 15× speedup.

This evaluation demonstrates that while supporting more complex and flexible quantization schemes, T-MAN also maintains comparable performance for prefill and superior performance for decoding.

6.4 Power and Energy Consumption

A key advantage of T-MAN is its exclusive execution on the power-efficient NPU, eliminating CPU involvement during both prefill and decoding phases. We measure power consumption by reading from /sys/class/power_supply at an interval of 100 ms during inference.

Framework	Prefill		Decoding	
	Power (W)	Energy (J/token)	Power (W)	Energy (J/token)
QNN-$W_{INT4}A_{INT16}$	4.96	**0.0073**	4.72	0.134
llm.npu-$W_{INT8}A_{INT8}$	8.89	0.0269	-	-
llm.npu-$W_{INT4}A_{INT8}$	-	-	8.31	0.612
bitnet.cpp-$W_{INT2}A_{INT8}$	8.22	0.196	8.22	0.490
T-MAN-$W_{INT2}A_{INT16}$	5.01	**0.0080**	4.91	**0.101**

Table 3. Comparison of power and energy efficiency of BitNet-2B on Snapdragon 8 Gen 3.

Compared to the CPU-only bitnet.cpp, T-MAN reduces average power consumption by 40%. When combined with the significant performance improvements, this translates to energy consumption reductions of 24.5× for prefill and 4.9× for decoding phases. Compared to the hybrid NPU-CPU llm.npu, T-MAN reduces power consumption by 45% for prefill. The unified NPU execution eliminates the power overhead of maintaining active CPU cores for outlier computations, resulting in overall energy savings of 71% for prefill and 84% for decoding. Although both T-MAN and QNN utilize NPU-only computation, T-MAN still achieves 25% energy reduction for decoding due to inference speedup.

6.5 Ablation Study

We conduct an ablation study to quantify the benefits of: (1) the efficient LUT-based dequantization kernel, and (b) the pipelined execution model.

6.5.1 Efficiency of the LUT-dequant kernel. We benchmark our LUT-based dequantization kernel against two baselines: (a) *LoadFull*, which loads pre-converted full-precision weights directly from memory, and (b) *ConvertDQ*, which uses the NPU's standard floating-point operations for dequantization. As shown in Fig. 16, our kernel achieves a 10.2×

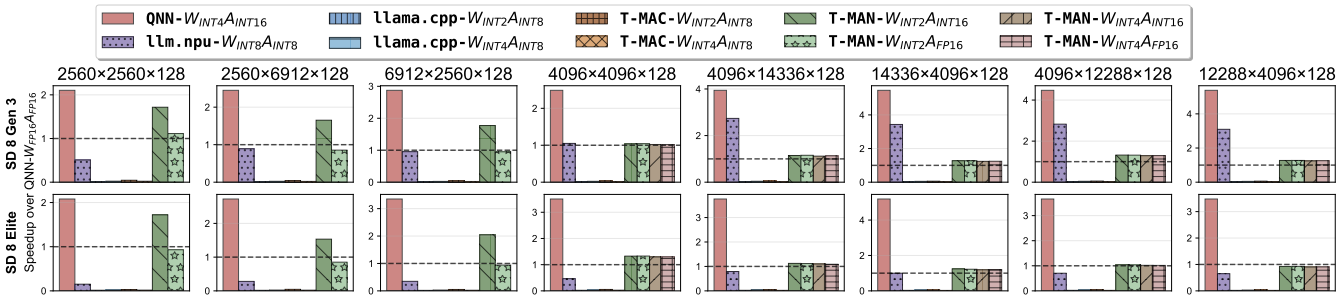


Figure 13. mpGEMM performance comparison. A sequence length of 128 is chosen. It is selected to align with the optimal performance settings for chunked prefill adopted by QNN and llm.npu.

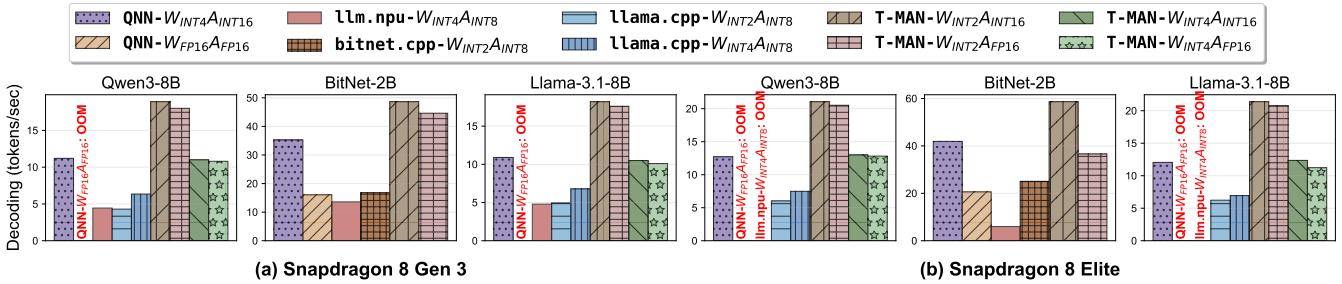


Figure 14. Decoding throughput comparison

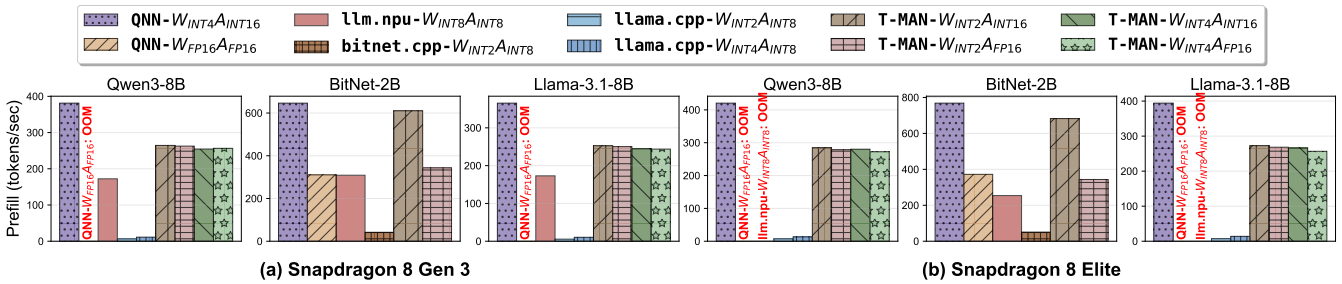


Figure 15. Prefill throughput comparison

speedup over *ConvertDQ* by avoiding inefficient float conversion instructions. It also outperforms *LoadFull* by 4.9×. This latter gain is attributed to reduced pressure on main DDR memory, as our method loads compact quantized weights and stores the full-precision results in on-chip memory.

6.5.2 Gain from pipelined execution. Next, we evaluate the performance gain from our three-stage pipeline: (1) **DMA** loads quantized weights, (2) **DQ** dequantizes them into on-chip memory, and (3) **MM** performs the matrix multiplication. As illustrated in Fig. 17, pipelined execution achieves a 1.5× speedup over a sequential approach. The overhead introduced by T-MAN is only 10% over the matmul stage alone, confirming that our pipeline effectively hides the latency of weight loading and dequantization.

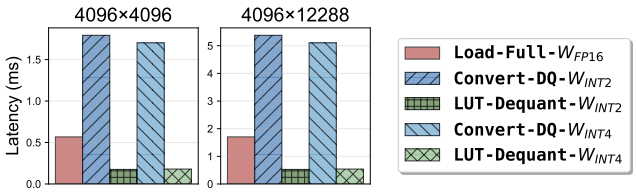


Figure 16. Latency comparison of different methods to prepare full-precision weights on Snapdragon 8 Gen 3.

6.6 Accuracy Benchmark

A key advantage of T-MAN is its support for per-block quantization schemes. We compare its perplexity (PPL) on the

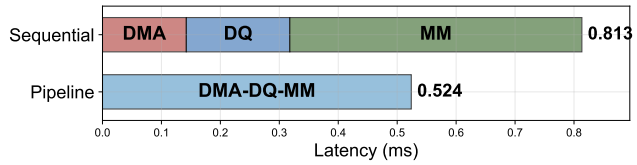


Figure 17. Comparison of sequential vs. pipelined execution for a $4096 \times 4096 \times 128$ W_{INT4} GEMM on Snapdragon 8 Gen3.

Framework	Llama-3.1-8B-Instruct PPL ↓	Qwen3-8B PPL ↓
QNN-$W_{INT4}A_{INT16}$	18.62	25.37
T-MAN-$W_{INT2}A_{INT16}$	**12.81**	**13.14**

Table 4. Perplexity on WikiText2. T-MAN with per-block W_{INT2} outperforms per-channel W_{INT4} used by QNN.

WikiText2 dataset against QNN, which is limited to per-tensor and per-channel quantization. Table 4 shows that T-MAN not only delivers better throughput but also achieves significantly lower perplexity. Even when using a lower bit-width, T-MAN achieves perplexity reduction of 48.2% and 31.7% for Qwen3-8B and Llama-3.1-8B-Instruct respectively.

7 Discussion and Limitations

Potential for hardware-accelerated mpGEMM. While the Qualcomm NPU lacks native 2-bit support, its existing 4-bit matrix multiplication kernels offer a viable path to accelerate the prefill stage of per-tensor quantized BitNet. Our proposed LUT-dequantization method can be applied to dequantize 2-bit weights to 4-bit on-the-fly. However, the implementation of such a custom kernel is currently infeasible, as the hardware instructions for low-bit matrix operations are not yet exposed to developers. The alternative approach, leveraging high-level 4-bit operations via the QNN framework, is constrained by its reliance on a black-box offline weight packing process.

Long-context attention. Our evaluations are conducted on a context length of 1024 due to the absence of an efficient attention kernel implementation for longer sequences on the NPU. The end-to-end speedup of T-MAN is also bottlenecked by the attention. While the optimization of attention was outside the scope of this paper, it represents a critical area for future work. Such efforts will become more practical as the NPU's matrix core architecture becomes more accessible to developers.

Different NPU hardware. Although T-MAN was developed and tested on Qualcomm NPUs, its underlying principles are broadly applicable to other NPUs, most of which feature a similar architecture of vector and matrix cores and limited FLOPs. A primary limitation, however, is hardware programmability. Certain platforms, such as Apple NPUs, are closed systems accessible only through high-level APIs, precluding the implementation of custom kernels. Adapting our design to other programmable hardware, like the Ascend NPU [1], requires further investigation on how to efficiently implement the lookup operation and remains an avenue for future exploration.

8 Related Works

LUT for low-bit inference. An emerging optimization for low-bit models is using LUT to eliminate dequantization and replace expensive multiplications with table lookups. This approach has been applied across hardware, starting with CNNs on CPUs (DeepGEMM [16]). For LLMs on GPUs, methods like LUT-GEMM [29] and FLUTE [19] developed custom kernels to operate directly on quantized weights, with LUT Tensor Core [26] proposing a software-hardware co-design for further efficiency. T-MAC [38] pioneers an efficient LUT-based method on CPUs to achieve linear speedup with bit-width reduction.

NPU for LLM inference. Recent research has increasingly focused on leveraging NPUs to address bottlenecks in LLM inference, often by creating heterogeneous systems. For instance, Hybe [27] proposes a GPU-NPU hybrid system, assigning the prefill stage to GPUs and the decoding stage to NPU. The NPU adopts a lightweight architecture specifically designed for decoding. For on-device inference, llm.npu [40] presents a hybrid NPU-CPU system where the NPU handles most of the prefill computation while the CPU manages outlier calculations in parallel. Another approach combines NPUs with PIM. NeuPIMs [20] and IANUS [34] exemplify this by using NPUs for GEMM and PIM for GEMV. NeuPIMs focuses on enabling concurrent execution, while IANUS introduces a unified memory system to minimize data movement. Beyond architecture, V10 [42] improves NPU utilization in cloud inference through a hardware-assisted multi-tenancy framework that enables fine-grained resource sharing and scheduling.

On-device optimization of LLM. Beyond model quantization and hardware acceleration, algorithmic optimizations are crucial for deploying LLMs on-device. Knowledge distillation is used to create smaller student models by transferring knowledge from larger teacher models [21], and can also be applied QAT to mitigate quantization errors [13]. To accelerate inference, speculative decoding breaks the sequential nature of token generation. This technique uses a small draft model to propose candidate tokens that are verified in parallel by the main model. Variations on this approach, such as adding extra decoding heads [10] or using lookahead strategies [15], further enhance decoding speed and eliminate the need for the draft model.

9 Conclusion

This paper realized the first end-to-end LLM inference on the NPUs, achieved the SOTA latency and energy among current on-device inference systems. It is based on the key idea to use table lookups to subsume operations not well supported by the highly specialized hardware. The 1.4× and 3.1× speedup for prefill and decoding, and the energy and memory saving demonstrate the effectiveness of T-MAN.

References

[1] 2025. Huawei Ascend. https://www.hiascend.com.

[2] accessed 2024. llama.cpp. https://github.com/ggerganov/llama.cpp.

[3] accessed 2024. MICROSOFT BitBLAS. https://github.com/microsoft/bitblas.

[4] accessed 2025. Neural Processing Unit Market Report 2026–2033: Innovations, Opportunities & Regional Trends. https://www.linkedin.com/pulse/neural-processing-unit-market-report-20262033-innovations-p5voc/.

[5] accessed 2025. ONNX Runtime QNN Execution Provider. https://onnxruntime.ai/docs/execution-providers/QNN-ExecutionProvider.html.

[6] accessed 2025. PowerServe: high-speed and easy-use LLM serving framework for local deployment. https://github.com/powerserve-project/PowerServe.

[7] accessed 2025. PyTorch Qualcomm AI Engine Backend. https://docs.pytorch.org/executorch/stable/backends-qualcomm.html.

[8] Apple. 2025. Apple Intelligence Foundation Language Models: Tech Report 2025. arXiv:2507.13575 [cs.LG] https://arxiv.org/abs/2507.13575

[9] Apple. 2025. Introducing Apple Foundation Models. https://machinelearning.apple.com/research/introducing-apple-foundation-models.

[10] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. *arXiv preprint arXiv: 2401.10774* (2024).

[11] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] https://arxiv.org/abs/2501.12948

[12] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. arXiv:2208.07339 [cs.LG] https://arxiv.org/abs/2208.07339

[13] Dayou Du, Yijia Zhang, Shijie Cao, Jiaqi Guo, Ting Cao, Xiaowen Chu, and Ningyi Xu. 2024. BitDistiller: Unleashing the Potential of Sub-4-Bit LLMs via Self-Distillation. arXiv:2402.10631 [cs.CL]

[14] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).

[15] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057* (2024).

[16] Darshan C Ganji, Saad Ashfaq, Ehsan Saboori, Sudhakar Sah, Saptarshi Mitra, Mohammadhossein Askarihemmat, Alexander Hoffman, Ahmed Hassanien, and Mathieu Leonardon. 2023. DeepGEMM: Accelerated Ultra Low-Precision Inference on CPU Architectures using Lookup Tables. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4655–4663.

[17] Google. accessed 2025. Gemma 3 on Mobile and Web With Google AI Edge. https://developers.googleblog.com/en/gemma-3-on-mobile-and-web-with-google-ai-edge/.

[18] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[19] Han Guo, William Brandon, Radostin Cholakov, Jonathan Ragan-Kelley, Eric P. Xing, and Yoon Kim. 2025. Fast Matrix Multiplications for Lookup Table-Quantized LLMs. arXiv:2407.10960 [cs.LG] https://arxiv.org/abs/2407.10960

[20] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 722–737. doi:10.1145/3620666.3651380

[21] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes. arXiv:2305.02301 [cs.CL] https://arxiv.org/abs/2305.02301

[22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

[23] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. 2023. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978* (2023).

[24] Shuming Ma, Hongyu Wang, Shaohan Huang, Xingxing Zhang, Ying Hu, Ting Song, Yan Xia, and Furu Wei. 2025. BitNet b1.58 2B4T Technical Report. arXiv:2504.12285 [cs.CL] https://arxiv.org/abs/2504.12285

[25] Microsoft. accessed 2025. Phi Silica Small But Mighty on Device SLM. https://blogs.windows.com/windowsexperience/2024/12/06/phi-silica-small-but-mighty-on-device-slm/.

[26] Zhiwen Mo, Lei Wang, Jianyu Wei, Zhichen Zeng, Shijie Cao, Lingxiao Ma, Naifeng Jing, Ting Cao, Jilong Xue, Fan Yang, and Mao Yang. 2025. LUT Tensor Core: A Software-Hardware Co-Design for LUT-Based Low-Bit LLM Inference. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 514–528. doi:10.1145/3695053.3731057

[27] Seungjae Moon, Junseo Cha, Hyunjun Park, and Joo-Young Kim. 2025. Hybe: GPU-NPU Hybrid System for Efficient LLM Inference with Million-Token Context Window. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 808–820. doi:10.1145/3695053.3731051

[28] OpenAI. 2024. OpenAI o1 System Card. arXiv:2412.16720 [cs.AI] https://arxiv.org/abs/2412.16720

[29] Gunho Park, Baeseong Park, Minsub Kim, Sungjae Lee, Jeonghoon Kim, Beomseok Kwon, Se Jung Kwon, Byeongwook Kim, Youngjoo Lee, and Dongsoo Lee. 2023. LUT-GEMM: Quantized Matrix Multiplication based on LUTs for Efficient Inference in Large-Scale Generative Language Models. arXiv:2206.09557 [cs.DC]

[30] Open Compute Project. accessed 2025. MXFP4. https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf.

[31] Qualcomm. accessed 2025. Qualcomm® AI Engine Direct software development kit (aka the QNN SDK). https://docs.qualcomm.com/bundle/publicresource/topics/80-63442-50/introduction.html.

[32] Qualcomm. accessed 2025. Snapdragon Summit's AI highlights: A look at the future of on-device AI. https://www.qualcomm.com/news/onq/2024/10/snapdragon-summit-ai-highlights-a-look-at-the-future-of-on-device-ai/.

[33] Qualcomm. accessed 2025. Snapdragon X Elite. https://www.qualcomm.com/products/mobile/snapdragon/laptops-

and-tablets/snapdragon-x-elite/.

[34] Minseok Seo, Xuan Truong Nguyen, Seok Joong Hwang, Yongkee Kwon, Guhyun Kim, Chanwook Park, Ilkon Kim, Jaehan Park, Jeongbin Kim, Woojae Shin, Jongsoon Won, Haerang Choi, Kyuyoung Kim, Daehan Kwon, Chunseok Jeong, Sangheon Lee, Yongseok Choi, Wooseok Byun, Seungcheol Baek, Hyuk-Jae Lee, and John Kim. 2024. IANUS: Integrated Accelerator based on NPU-PIM Unified Memory System. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 545–560. doi:10.1145/3620666.3651324

[35] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[36] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. 2023. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453* (2023).

[37] Jinheng Wang, Hansong Zhou, Ting Song, Shijie Cao, Yan Xia, Ting Cao, Jianyu Wei, Shuming Ma, Hongyu Wang, and Furu Wei. 2025. Bitnet.cpp: Efficient Edge Inference for Ternary LLMs. arXiv:2502.11880 [cs.LG] https://arxiv.org/abs/2502.11880

[38] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. 2025. T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 278–292. doi:10.1145/3689031.3696099

[39] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.

[40] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. 2025. Fast On-device LLM Inference with NPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 445–462. doi:10.1145/3669940.3707239

[41] Yuzhuang Xu, Xu Han, Zonghan Yang, Shuo Wang, Qingfu Zhu, Zhiyuan Liu, Weidong Liu, and Wanxiang Che. 2024. OneBit: Towards Extremely Low-bit Large Language Models. *arXiv preprint arXiv:2402.11295* (2024).

[42] Yuqi Xue, Yiqi Liu, Lifeng Nai, and Jian Huang. 2023. V10: Hardware-Assisted NPU Multi-tenancy for Improved Resource Utilization and Fairness. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) *(ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 24, 15 pages. doi:10.1145/3579371.3589059